

DSC1105_MONFERO_FA4

John Benedict A. Monfero

2025-02-28

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2     3.5.1      ✓ tibble     3.2.1
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr       1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## ⓘ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

1. Using the Mortality by Latitude data

```
data <- read.csv(
  "mortality_by_latitude.csv" # file path matters
)
```

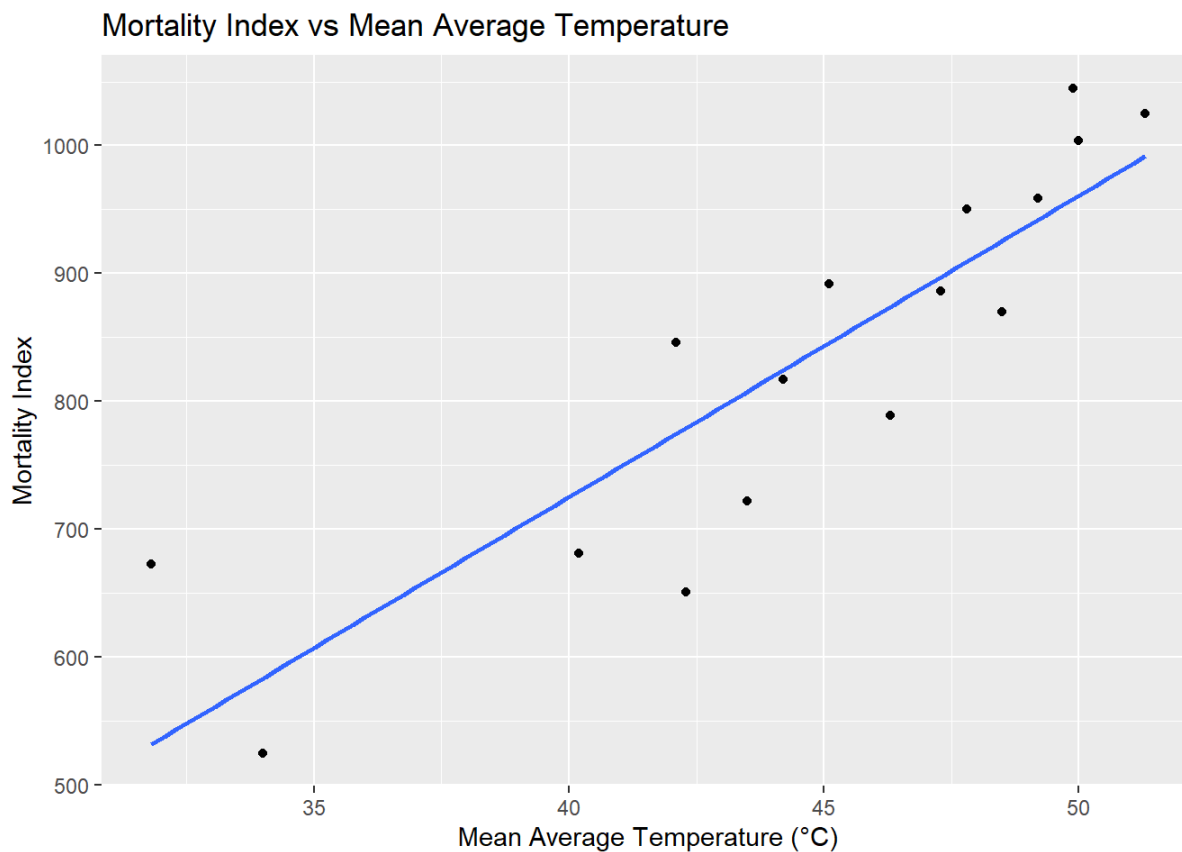
```
# Preview the dataset
data
```

```
##   latitude mortality_index temperature
## 1       50          1025          51.3
## 2       51          1045          49.9
## 3       52          1004          50.0
## 4       53           959          49.2
## 5       54           870          48.5
## 6       55           950          47.8
## 7       56           886          47.3
## 8       57           892          45.1
## 9       58           789          46.3
## 10      59           846          42.1
## 11      60           817          44.2
## 12      61           722          43.5
## 13      62           651          42.3
## 14      63           681          40.2
## 15      69           673          31.8
## 16      70           525          34.0
```

1.1. Make a plot of mortality index against mean average temperature.

```
data %>%
  ggplot(
    mapping = aes(
      x = temperature, y = mortality_index
    )
  ) +
  geom_point() +
  geom_smooth(
    method = "lm", se = FALSE
  ) +
  labs(
    title = "Mortality Index vs Mean Average Temperature",
    x = "Mean Average Temperature (°C)",
    y = "Mortality Index"
  )
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



1.2. Is it hollow up or hollow down?

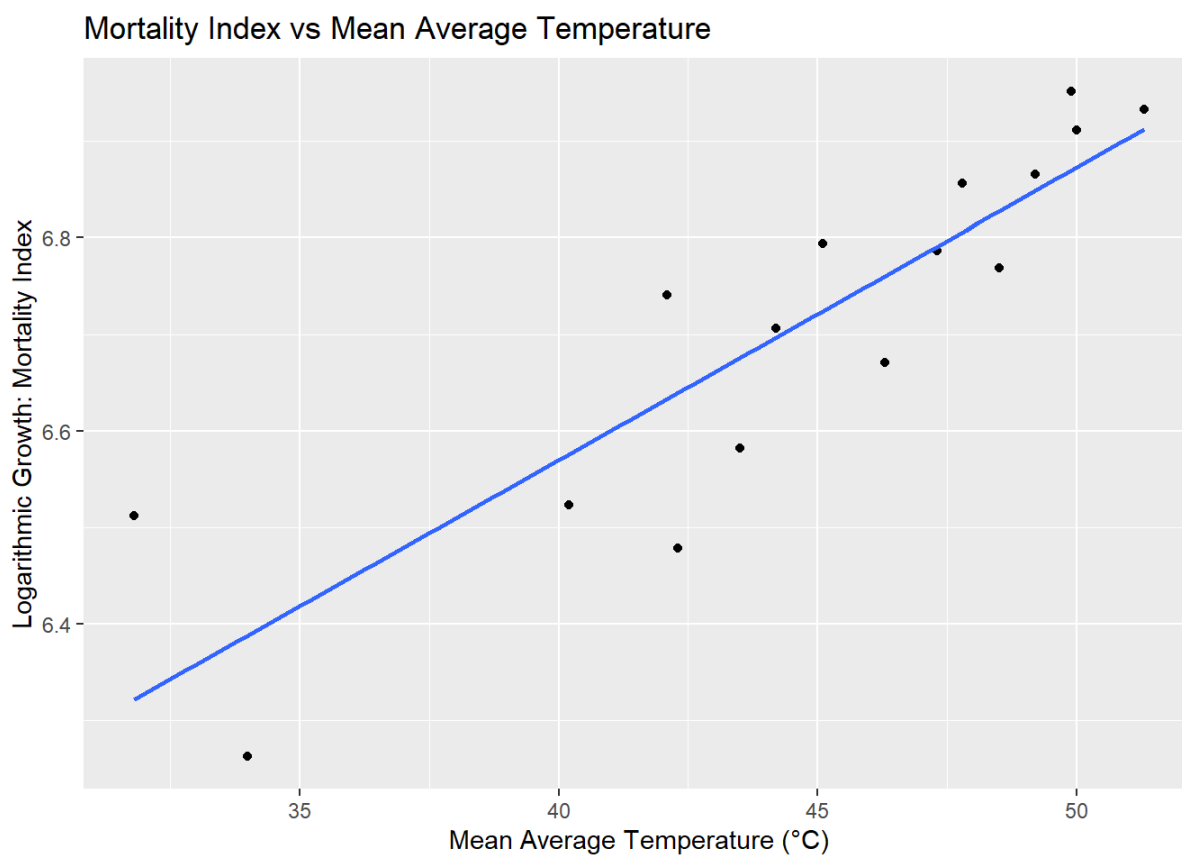
The given plot above shows up a trend - blue line - to be going upward as the data points follow through, indicating there exists a relationship that is hollow up!

1.3. Transformation to Straighten Out the Relationship

Instead of using the `mortality_index`, we will apply logarithmic transformation of `mortality_index` as a new basis: `log(data$mortality_index)` will be utilized to further demonstrate stronger relationship.

```
data %>%
  ggplot(
    mapping = aes(
      x = temperature, y = log(mortality_index)
    )
  ) +
  geom_point() +
  geom_smooth(
    method = "lm", se = FALSE
  ) +
  labs(
    title = "Mortality Index vs Mean Average Temperature",
    x = "Mean Average Temperature (°C)",
    y = "Logarithmic Growth: Mortality Index"
  )
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



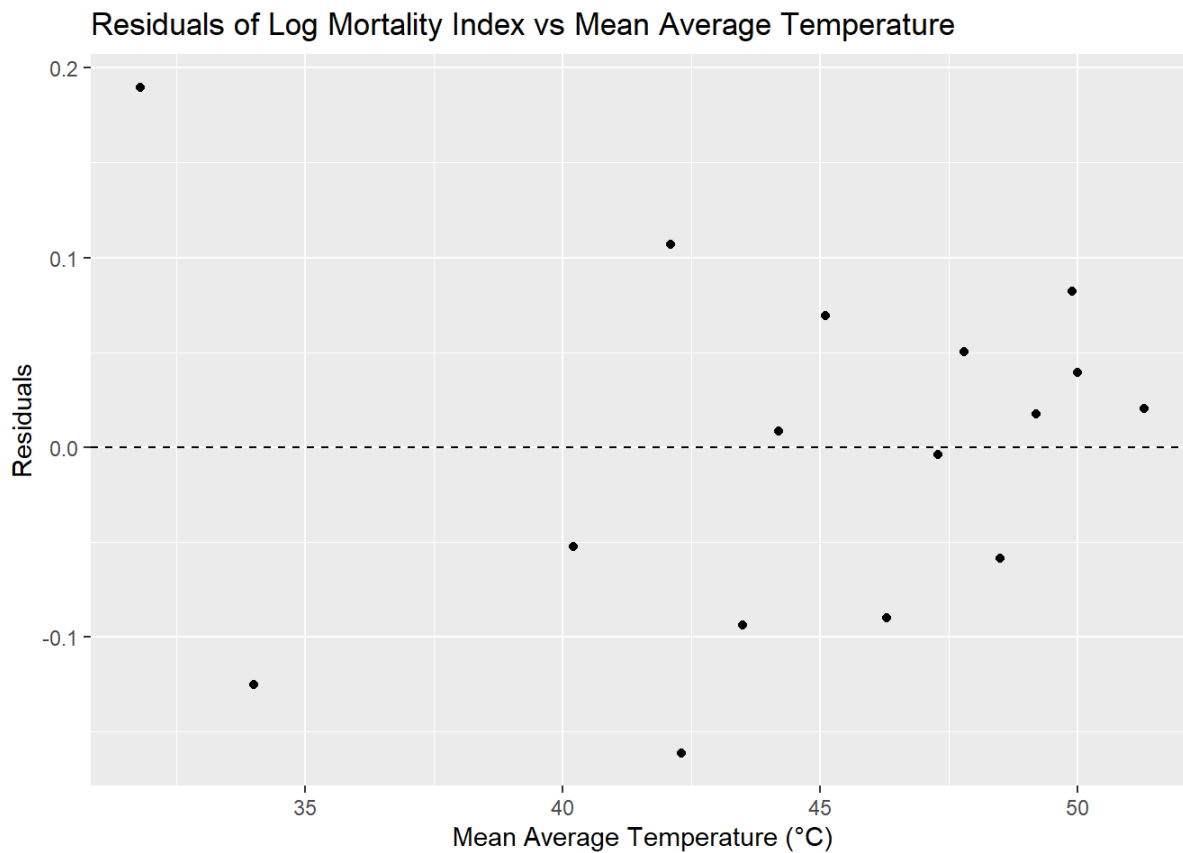
1.4. Checking the residuals remains using the `log(mortality_index)`

Residual: A residual value is **a measure of how much a regression line vertically misses a data point**. Regression lines are the best fit of a set of data. You can think of the lines as averages; a few data points will fit the line and others will noticeably miss.

```
residuals <- resid(
  lm(
    log(
      mortality_index
    ) ~ temperature,
    data = data
  )
)
```

Plotting the residuals

```
data %>%
  ggplot(
    mapping = aes(
      x = temperature, y = residuals
    )
  ) +
  geom_point() +
  geom_hline(
    yintercept = 0.00, linetype = "dashed"
  ) +
  labs(
    title = "Residuals of Log Mortality Index vs Mean Average Temperature",
    x = "Mean Average Temperature (°C)",
    y = "Residuals"
  )
```



```
print("The figure above implies that as we increase the mean average temperature, the trend with the mortality index became more fitting to the trend line, and the missing scattered points became less obvious.")
```

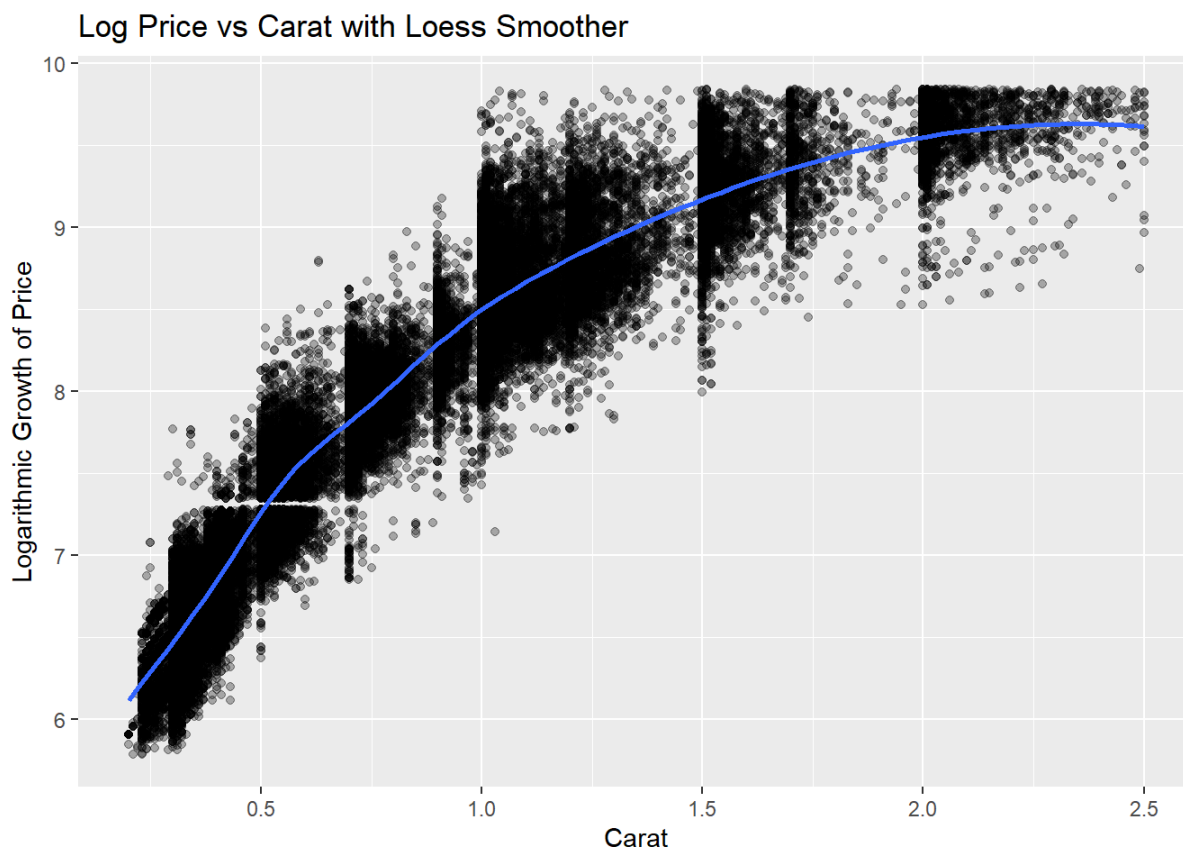
```
## [1] "The figure above implies that as we increase the mean average temperature, the trend with the mortality index became more fitting to the trend line, and the missing scattered points became less obvious."
```

2. Diamond Dataset

2.1. Plot of Log Price as a Function of Carat with a Loess Smoother

```
diamonds %>%  
  filter(carat <= 2.50  
    # Use the value `5.00` if none entries shall be filtered out  
  ) %>%  
  ggplot(  
    mapping = aes(  
      x = carat, y = log(price)  
    )  
  ) +  
  geom_point(  
    alpha = 0.3  
  ) +  
  geom_smooth(  
    method = "loess", span = 0.5, se = FALSE  
  ) +  
  labs(title = "Log Price vs Carat with Loess Smoother",  
    x = "Carat",  
    y = "Logarithmic Growth of Price")
```

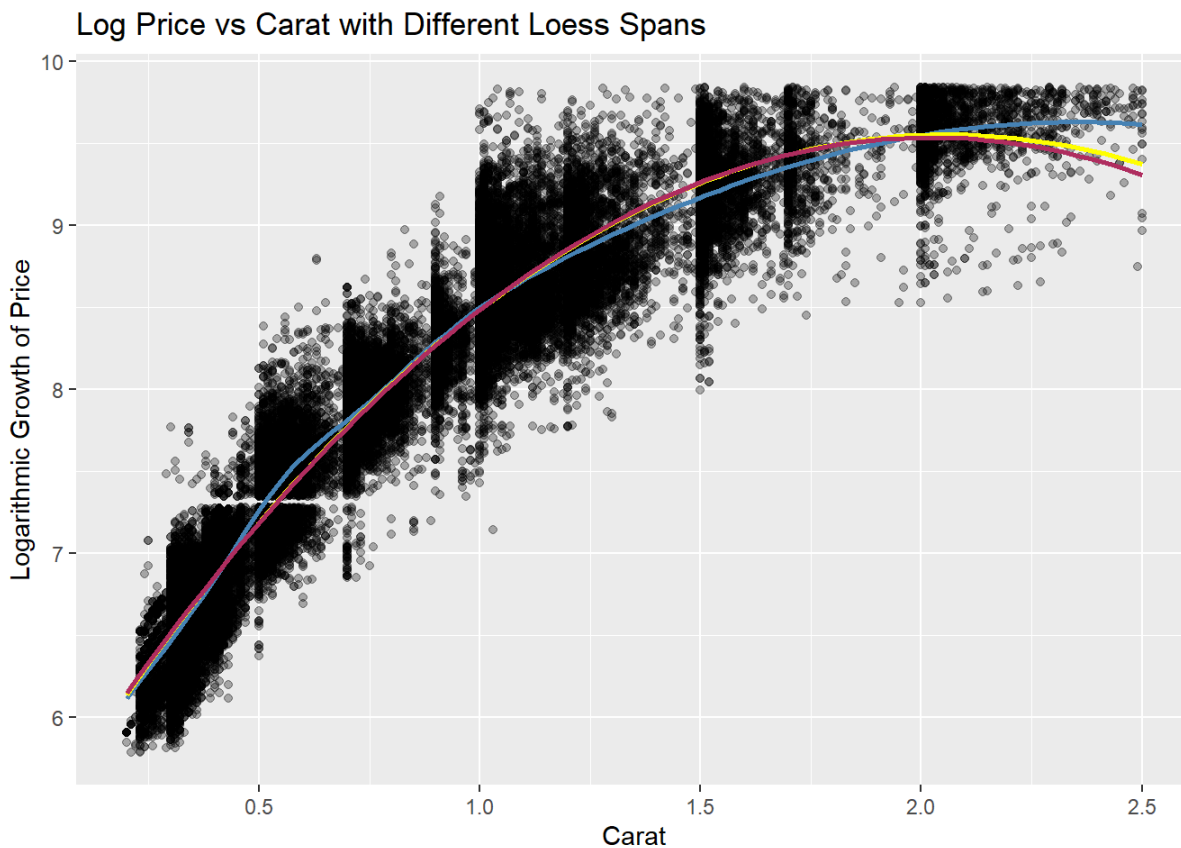
```
## `geom_smooth()` using formula = 'y ~ x'
```



2.2. Trying Several Values for Span and Degree Arguments

```
diamonds %>%
  filter(carat <= 2.50
    # Use the value `5.00` if none entries shall be filtered out
  ) %>%
  ggplot(
    mapping = aes(
      x = carat, y = log(price)
    )
  ) +
  geom_point(
    alpha = 0.3
  ) +
  geom_smooth(
    method = "loess", span = 0.5, se = FALSE, color = "steelblue"
  ) +
  geom_smooth(
    method = "loess", span = 1.5, se = FALSE, color = "yellow"
  ) +
  geom_smooth(
    method = "loess", span = 2.5, se = FALSE, color = "maroon"
  ) +
  labs(title = "Log Price vs Carat with Different Loess Spans",
    x = "Carat",
    y = "Logarithmic Growth of Price")
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



As `span` increases for `LOESS`, it follows more smoother fit that overgeneralized accross the data, risks of overfitting could happened. Too much low `span` for `LOESS` could underfit and maximizing the total area for residuals of the function.

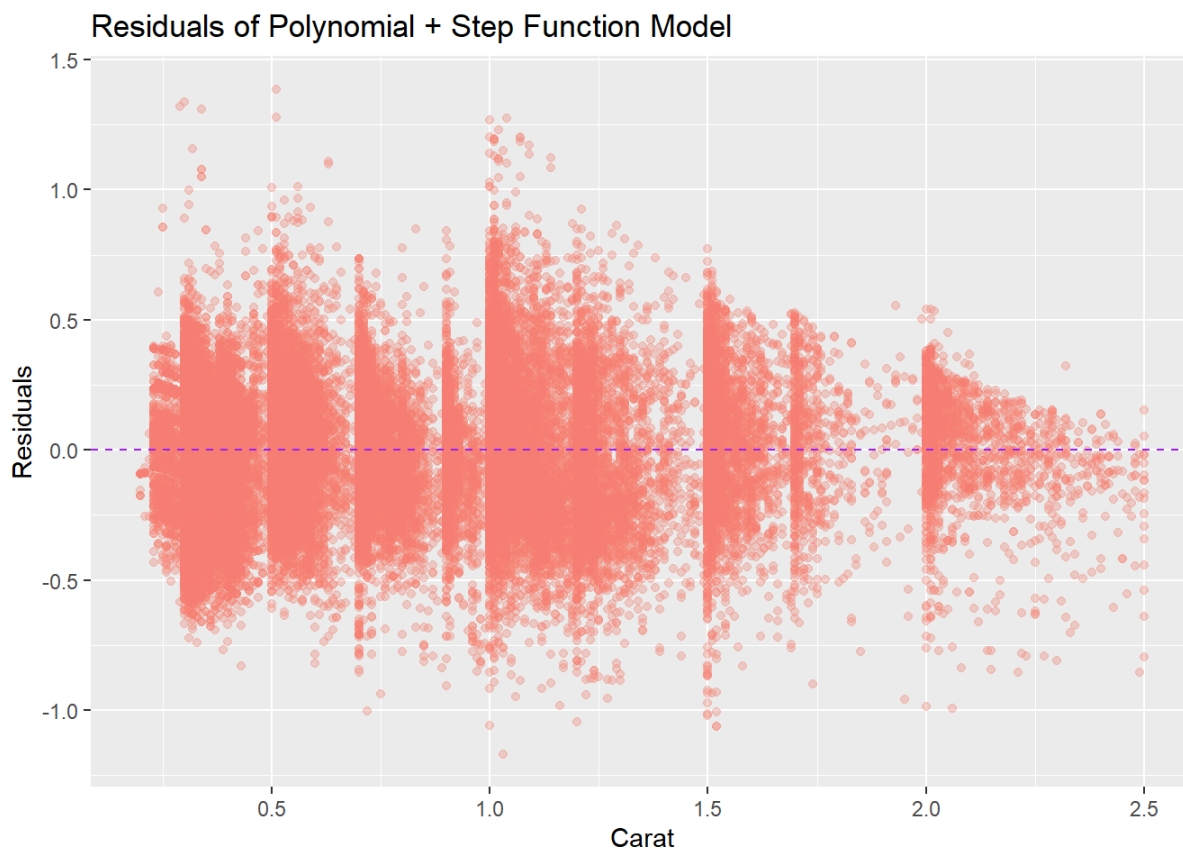
Hence, a span of 0.5 above shall be chosen as it provides a good balance between smoothness and capturing the necessary trend in the data

3. Comparing Loess Smoother to Polynomial + Step Function Regression

3.1. Plot of Residuals for Both Models

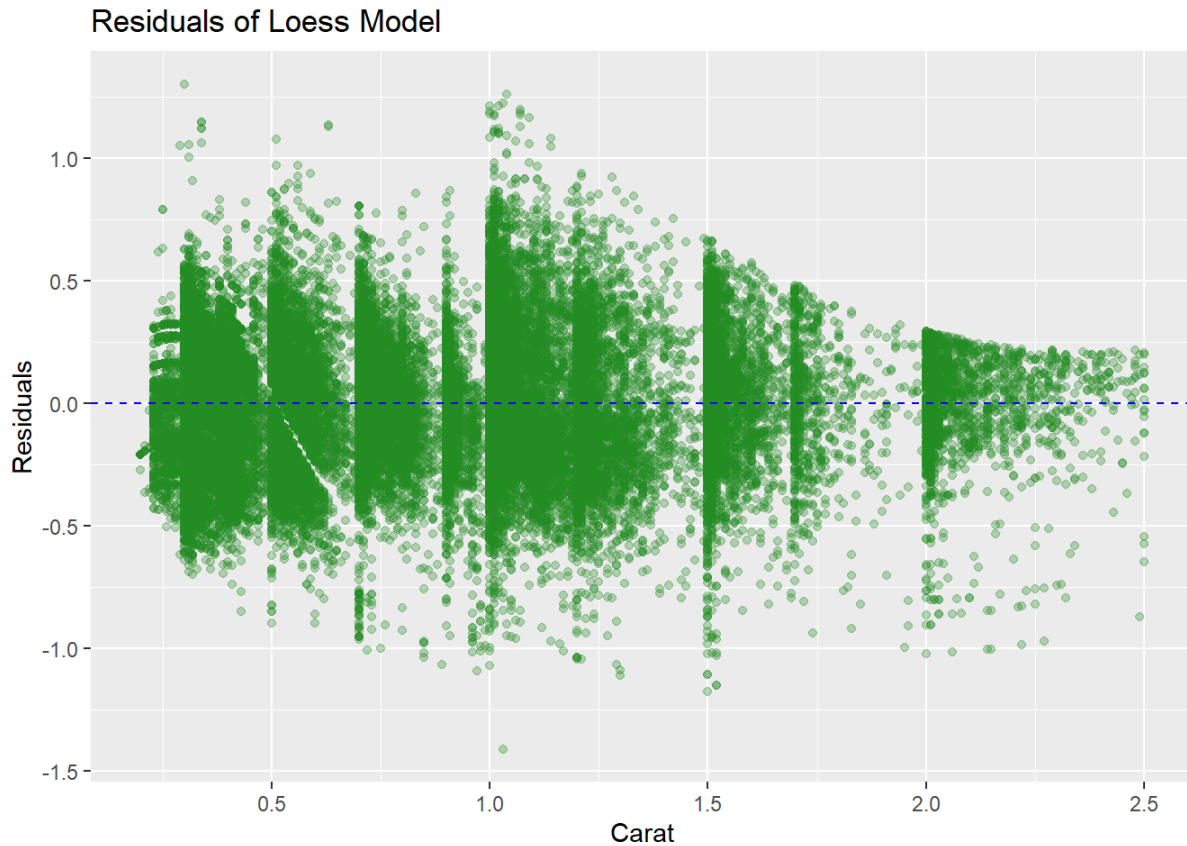
3.1.1. Polynomial + step function regression

```
subset_diamonds <- diamonds %>%  
  filter(carat <= 2.50)  
model_poly <- lm(log(price) ~ poly(carat, 3) + cut, data = subset_diamonds)  
residuals_poly <- resid(model_poly)  
# Plotting residuals for polynomial model  
ggplot(subset_diamonds, aes(x = carat, y = residuals_poly)) +  
  geom_point(alpha = 0.3, color = "salmon") +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "purple") +  
  labs(title = "Residuals of Polynomial + Step Function Model",  
       x = "Carat",  
       y = "Residuals")
```



3.1.2. Plotting residuals for loess model

```
model_loess <- loess(log(price) ~ carat, data = subset_diamonds, span = 0.5)
residuals_loess <- residuals(model_loess)
# Plotting residuals for Loess model
ggplot(subset_diamonds, aes(x = carat, y = residuals_loess)) +
  geom_point(alpha = 0.3, color = "forestgreen") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "blue") +
  labs(title = "Residuals of Loess Model",
       x = "Carat",
       y = "Residuals")
```



3.2. Which model is more faithful

The 3.1.2 loess model appears to be more faithful to the data as it captures the non-linear trends better and has overall smaller residuals.